Lecture 4
Branches: Parallel Lines of Work

`git branch`, `git switch`, `HEAD`, and why branches are cheap

Learning goals

- Explain what a branch is: a movable label pointing at a commit.
- Explain what `HEAD` is, and how it moves when you commit or switch.
- Create, list, switch, rename and delete branches.
- Read the history graph of several branches.
- Handle uncommitted changes when you switch branches.

1 Why branches?

Imagine you are writing a report that is in good shape. You want to try a risky new structure, but you do not want to wreck the good version, and a friend wants to fix typos at the same time. In Lecture 0 we met the old habit of copying the folder (`report_v2_final`). A **branch** is Git's clean answer: a separate line of work inside the same repository. You can experiment freely on a branch, and the stable work stays untouched until you decide to combine them (Lecture 5).

Key idea

A branch lets you work on an idea *in isolation*. If the idea is good you keep it; if it is bad you drop it. Either way the main line of work was never at risk.

Every repository already has one branch, usually named `main` (the name you set in Homework 0.3), which `git init` created. In this lecture we create more and move between them. We use a practice repository with the same history on every computer. From the course folder run:

```
$ bash codeHints/L04/setup_demo4.sh
$ cd practice/demo4
```

It has three commits on `main`. (To get a fresh copy at any time: leave the folder with `cd ../..`, run the script again with `--reset`, then `cd practice/demo4`.)

2 A branch is just a label

This is the most important idea of the lecture, and it is simpler than it sounds. A branch is *not* a copy of your files. It is a small label, a **pointer**, that holds the ID of one commit.

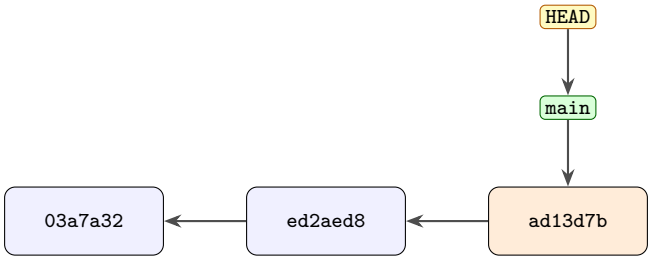


Figure 1: Three commits. The branch `main` points at the newest one. `HEAD` points at the branch you are on.

```
$ git log --oneline --decorate
ad13d7b (HEAD -> main) Add crepe recipe
ed2aed8 Add pancake recipe
03a7a32 Add README
```

The words in brackets are the labels attached to a commit: `HEAD -> main` means "`HEAD` points at `main`, which points here". (Your terminal shows these automatically; if it does not, add `--decorate`.)

In Lecture 3 we said `HEAD` is "the commit you are on". Now the full story: **`HEAD` points at the *current branch*, and the branch points at a commit.** When you make a commit, Git creates the new commit and moves the *current branch* label forward to it. Nothing else moves.

Quick fact

Because a branch is only a 40-character commit ID in a tiny file, creating one takes a few milliseconds and almost no space, however large the project is. In some older systems a branch meant copying the whole project, which is why branching was slow and rare. In Git it is so cheap that you should use branches freely.

3 Creating and switching

```
$ git branch
* main
$ git branch waffles
$ git branch
* main
  waffles
```

`git branch` lists the branches; the star marks the one you are on. `git branch waffles` creates a new branch *at the commit you are on* but does not move you. Both labels now point at the same commit:

```
$ git log --oneline --decorate
ad13d7b (HEAD -> main, waffles) Add crepe recipe
ed2aed8 Add pancake recipe
03a7a32 Add README
```

To move onto the new branch, `switch` to it:

```
$ git switch waffles
Switched to branch 'waffles'
```

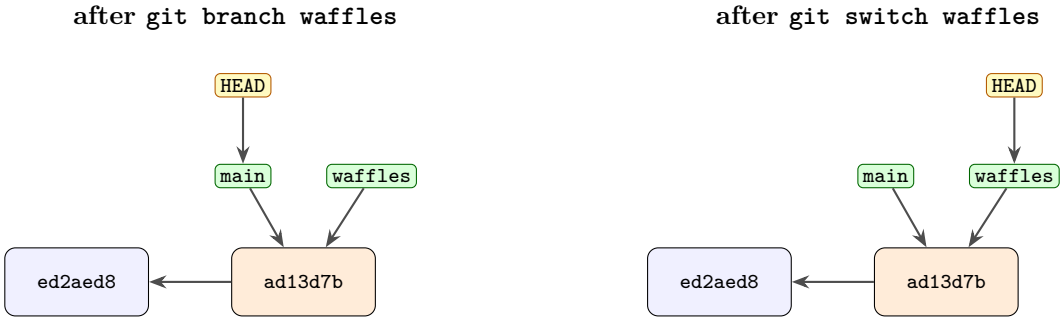


Figure 2: Creating a branch adds a second label to the same commit. Switching moves only `HEAD`. No files are copied.

The two steps are so common that there is a shortcut: `git switch -c waffles` creates the branch and switches to it in one go. To see which branch you are on, `git status` says `On branch ...` on its first line, and `git branch --show-current` prints just the name.

Quick fact

`git switch` arrived in Git 2.23 (2019). Before that, one command did several jobs: `git checkout waffles` switched branches and `git checkout -b waffles` created one. You will meet `checkout` in many tutorials; for branches it does the same as `switch`. We use `switch` because it does one thing only.

3.1 Choosing a name

Use short, lowercase names that describe the idea, with hyphens instead of spaces (spaces are not allowed): `waffles`, `fix-typo`, `add-login`. One branch per idea.

4 Committing on a branch

We are on `waffles`. Add a recipe and commit:

```
$ echo "Waffles: flour, milk, eggs, sugar" >> recipes.txt
$ git add recipes.txt
$ git commit -m "Add waffle recipe"
$ git log --oneline --decorate
12c4aa1 (HEAD -> waffles) Add waffle recipe
ad13d7b (main) Add crepe recipe
ed2aed8 Add pancake recipe
03a7a32 Add README
```

(The ID of any commit *you* make will differ from the one printed here, because it depends on the time of the commit. Only the three setup commits `03a7a32`, `ed2aed8` and `ad13d7b` are the same on every computer, and the IDs of later commits below, such as `12c4aa1` and `d3cf462`, will differ on yours.) Git created the commit and moved the *current* branch, `waffles`, forward. `main` did not move:

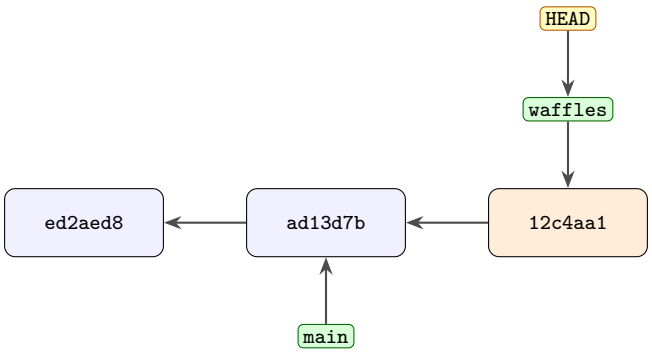


Figure 3: After one commit on `waffles`. Only the current branch label moved.

Switching changes your files

Your working directory always shows the files of the commit that the current branch points at. When you switch, Git rewrites your files to match:

```
$ cat recipes.txt          # on waffles
Pancakes: flour, milk, eggs
Crepes: flour, milk, eggs, butter
Waffles: flour, milk, eggs, sugar
$ git switch main
Switched to branch 'main'
$ cat recipes.txt          # the waffle line is gone
Pancakes: flour, milk, eggs
Crepes: flour, milk, eggs, butter
```

The waffle recipe is not lost: it is saved in the commit that `waffles` points to. Switch back and the file returns. This is the superpower of branches: one folder, several versions of the project, one command to change between them.

Try it

In `demo4`, switch between `main` and `waffles` a few times, each time running `cat recipes.txt` and `git status`. Watch the file change, and note that `git status` always names the current branch. **Finish on `main`** (check with `git branch --show-current`), because the next section continues from there.

5 Diverging history

Now work on `main` as well. Make sure you are on `main`, then add a muffin recipe:

```
$ echo "Muffins: flour, milk" >> recipes.txt
$ git add recipes.txt
$ git commit -m "Add muffin recipe"
$ git log --oneline --decorate --all --graph
* d3cf462 (HEAD -> main) Add muffin recipe
| * 12c4aa1 (waffles) Add waffle recipe
|/
* ad13d7b Add crepe recipe
* ed2aed8 Add pancake recipe
* 03a7a32 Add README
```

The history has *forked*. Both new commits have the crepe commit as their parent, but neither knows about the other.

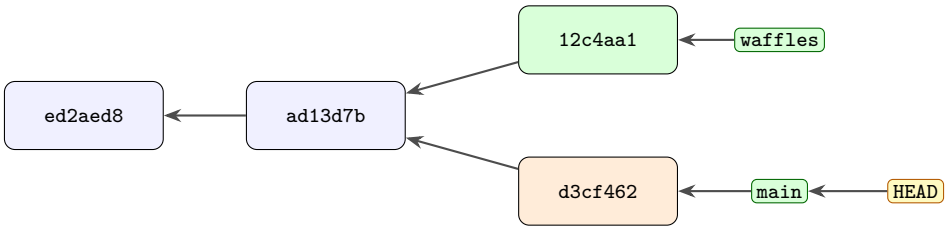


Figure 4: Two branches that have diverged. Each has a commit the other lacks. Lecture 5 combines them.

In the graph each `*` is a commit, lines show how commits connect, and labels in brackets are branch names. `--all` shows every branch (otherwise only the current one's history) and `--graph` draws the lines. Run this whenever you wonder "what does my history look like?".

```
$ git branch -v
* main    d3cf462 Add muffin recipe
  waffles 12c4aa1 Add waffle recipe
```

`git branch -v` lists each branch with the commit it points to.

Pitfall

The most common branch mistake is committing on the wrong branch because you forgot which one you were on. Before you commit, check the first line of `git status`. (If it happens, Lecture 8 shows how to move the commit.)

6 Switching with uncommitted changes

What happens if you have edited a file and switch? Git protects your work. If the branches have the *same* version of the file you edited, or you only added brand-new files, your changes simply come along. But if the target branch has a *different* version of a file you edited, Git refuses rather than overwrite your edit:

```
$ echo "Scones: flour, butter" >> recipes.txt
$ git switch waffles
error: Your local changes to the following files would be overwritten by checkout:
        recipes.txt
Please commit your changes or stash them before you switch branches.
Aborting
```

(Here "checkout" just means the file rewriting that a switch does. The wording varies a little between Git versions.)

Nothing has changed: you are still on `main`, and your edit is intact. (To continue with the lecture, discard the experiment with `git restore recipes.txt`.) In a real situation you have three options: *commit* the change (the usual choice), *discard* it with `git restore recipes.txt` if it was junk (Lecture 3), or save it for later with `git stash` (Lecture 10).

Key idea

Good habit: commit, or at least check `git status`, before you switch branches. A clean working directory makes switching always safe.

7 More branch commands

Command	What it does
`git switch -c name start`	Create a branch that starts at `start`, a commit ID or another branch (for example `ed2aed8`, or `main~1`, the parent of the tip of `main`; the `~` notation from Lecture 3 works on branch names too), and switch to it. Without `start` it begins at the current commit.
`git branch name start`	The same, but do not switch.
`git branch -m old new`	Rename a branch (**m**ove).
`git branch -d name`	Delete a branch, but only if its commits are already included in the branch you are on (Git calls this "merged").

```
$ git branch experiment ed2aed8
$ git branch -m experiment trial
$ git branch -d trial
Deleted branch trial (was ed2aed8).
$ git branch -d waffles
error: The branch 'waffles' is not fully merged.
If you are sure you want to delete it, run 'git branch -D waffles'.
```

The first deletion worked because `trial` points at a commit that is already part of the branch you are on (`main`), so nothing would be lost. `waffles` has a commit that exists nowhere else, so Git says it is "not fully merged" ("merged" means: its commits are already reachable from your current branch; Lecture 5 shows how to merge) and refuses.

Pitfall

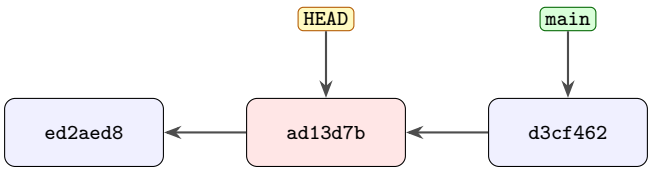
Deleting a branch deletes only the label, never the files in your folder. But commits that only that branch led to would have no label leading to them any more. That is why `-d` refuses to delete a branch with work that exists nowhere else, as above. `git branch -D` forces the deletion. Do not use `-D` until you understand why (Lecture 7 shows the one common safe exception; Lecture 10 shows how such commits can still be rescued for a while). For now, if Git refuses, believe it. (The exact wording of the error varies a little between Git versions.)

The usual life cycle is: create, work, merge (Lecture 5), delete with `-d`.

8 Detached HEAD: looking at an old commit

Sometimes you want to look at an old commit. `git switch --detach` puts `HEAD` on a commit directly, not on a branch:

```
$ git switch --detach HEAD~1
HEAD is now at ad13d7b Add crepe recipe
$ git log --oneline --decorate -2
ad13d7b (HEAD) Add crepe recipe
ed2aed8 Add pancake recipe
```



"detached": HEAD points at a commit, not a branch

Figure 5: Detached HEAD. Looking around is fine, but a commit made now belongs to no branch.

Git calls this a *detached HEAD*. You may reach it by accident with `git checkout <commit-id>`, where Git prints a long "detached HEAD" warning; it is the same state. It sounds alarming but is harmless as long as you only look. The danger: if you commit while detached, the new commit has no branch label, and when you switch away it becomes hard to find. To leave, just switch back: `git switch main`. If you *did* make useful commits while detached, give them a branch straight away: `git switch -c rescue`.

9 Homework 4

Homework 4: Branching out

From the course folder run:

```
$ bash codeHints/L04/setup_hw4.sh
$ cd practice/hw4
```

You get two commits on `main` and a misspelled branch. Look with `git log --oneline --graph --all --decorate`. Do the tasks in order; to start over, run `setup_hw4.sh --reset` from the course folder.

4.1 Branch
Create a branch `shopping` and switch to it.

4.2 Work on it
On `shopping`, create `shopping.txt` containing exactly the line `milk`; commit it as `Add shopping list`. Add a second line `eggs` (so the file has exactly the two lines `milk` and `eggs`) and commit that as `Add eggs`.

4.3 Back to main
Switch to `main`; check that `shopping.txt` is gone. Create `notes.txt` (any content) and commit it as `Add notes`.

4.4 Rename
Rename the misspelled branch `experimnt` to `experiment`.

4.5 Start point
Without switching, create a branch `hotfix` at `main~1`.

4.6 Finish and answer
End on `main` with a clean working directory. In `practice/hw4_answers.txt` (outside the repository; write it with `printf` or `echo`) write two lines:

```
branches=<how many branches the repository has now>
branch-is=<one word: what a branch is, technically>
```

Final graph: `main` and `shopping` fork at `Add app file`, where `hotfix` sits; `experiment` is on the first commit. **Check yourself:** from the course folder run `bash codeHints/L04/check_hw4.sh`. All `PASS`, then compare with the solutions at the end of this lecture (or in `solutions/L04/`).

10 Summary

- A branch is a movable label pointing at a commit; `HEAD` points at the current branch. Committing moves the current branch forward.
- `git branch` lists, `git branch name` creates, `git switch name` moves, `git switch -c name` does both; `-m` renames, `-d` deletes safely.
- Switching rewrites your files to match the branch. Commit first; Git refuses to overwrite uncommitted edits.
- `git log --oneline --decorate --all --graph` shows all branches and where they diverge.
- A detached `HEAD` (on a commit, not a branch) is for looking only. Further reading: *Pro Git*, Chapter 3.1.

Solutions

Stop here if you have not tried the homework yet.
Attempt every task first, run the checker, and only then read on.

Solutions to Homework 4

```
$ bash codeHints/L04/setup_hw4.sh
$ cd practice/hw4
$ git switch -c shopping
$ echo "milk" > shopping.txt
$ git add shopping.txt
$ git commit -m "Add shopping list"
$ echo "eggs" >> shopping.txt
$ git add shopping.txt
$ git commit -m "Add eggs"
$ git switch main
$ ls                          # no shopping.txt here
README.md  app.txt
$ echo "some notes" > notes.txt
$ git add notes.txt
$ git commit -m "Add notes"
$ git branch -m experimnt experiment
$ git branch hotfix main~1
$ git log --oneline --graph --all --decorate
* c6fac68 (HEAD -> main) Add notes
| * e7fa817 (shopping) Add eggs
| * 26c01df Add shopping list
|/
* 2e32674 (hotfix) Add app file
* 0a964a7 (experiment) Initial commit
$ printf 'branches=4\nbranch-is=pointer\n' > ../hw4_answers.txt
```

(The first two IDs, `0a964a7` and `2e32674`, are the same on your computer; the others depend on when you commit and will differ.)

4.1. `git switch -c shopping` creates and enters the branch at the `Add app file` commit, where `main` is at that moment. Creating it *before* the `Add notes` commit matters: the fork point is where the two lines of work separate.

4.2. Both commits go onto `shopping` because that is the branch `HEAD` points at. Check with `git status` (first line) before each commit.

4.3. After `git switch main` your files change: `shopping.txt` disappears from the folder. It is not lost; it lives in the commits of `shopping`. The `Add notes` commit moves `main` only. `shopping` stays where it was.

4.4. `git branch -m old new` renames. The branch still points at the first commit, so the checker's "experiment is the first commit" test passes.

4.5. `git branch hotfix main~1` creates the label at the parent of the newest `main` commit, which is `Add app file`, without switching. (`git switch -c` would also have created it, but you would then have to switch back.)

4.6. The four branches are `experiment`, `hotfix`, `main` and `shopping`. The word is **pointer**; "label" and "reference" are also accepted.

Common mistakes.

- Making the `shopping` commits while still on `main` (forgetting to switch). The checker then finds main with too many commits. Use `setup_hw4.sh --reset` and start again, watching the first line of `git status`.
- Creating `shopping` *after* the `Add notes` commit, so it forks from the wrong place.
- Using `git branch -D` or deleting `experimnt` instead of renaming it: the commit is still there but the branch name must be `experiment`.
- Ending the exercise on another branch: the last task asks you to be on `main`.